

4. Literature Survey

This section surveys the most relevant lane detection and backbone architectures considered during the design phase. Each entry includes the core idea, quantitative benchmarks where available, and an explicit rationale for selection or rejection in the context of this project.

4.1 SCNN — Spatial CNN (2018)

Pan et al., "Spatial As Deep: Spatial CNN for Traffic Scene Understanding," AAAI 2018.

Core Architecture Idea

SCNN applies slice-by-slice convolutions across rows and columns of a feature map, enabling information to propagate spatially across the full image. This allows the model to capture long-range structural dependencies — lane lines that span the full image height — which standard CNNs miss due to limited receptive fields.

Advantages

- Excellent at modelling long, continuous lane structures
- Robust to occlusion and lane marking gaps
- State-of-the-art on CULane and TuSimple at time of publication

Disadvantages

- Sequential slice convolutions cannot be parallelised on GPU — poor throughput
- High latency (~35ms per frame on a GTX 1080 Ti)
- Does not satisfy real-time requirements for Jetson Orin NX deployment

Benchmark Performance

CULane F1: 71.6%. TuSimple Accuracy: 96.5%. Inference speed: ~35ms on desktop GPU.

Verdict

REJECTED. Latency characteristics fundamentally incompatible with the ≥ 100 FPS deployment target. The sequential convolution design limits GPU utilisation to ~30% in benchmarks.

4.2 UFLD — Ultra Fast Lane Detection (2020)

Qin et al., "Ultra Fast Structure-Aware Deep Lane Detection," ECCV 2020.

Core Architecture Idea

UFLD reformulates lane detection as a row-based classification problem. The image is divided into horizontal row anchors; at each anchor, the model classifies which column bucket contains the lane or outputs a no-lane class. This reduces the prediction space from $H \times W$ pixel classifications (~73k for 360×640) to $\text{NUM_LANES} \times \text{NUM_ROWS} \times (\text{NUM_COLS} + 1) \approx 224$ values, achieving an order-of-magnitude speedup.

Advantages

- Extremely fast — 300+ FPS on a desktop GPU
- Structural loss term encodes prior knowledge that lanes are continuous
- Simple post-processing; compatible with TensorRT deployment

Disadvantages

- Struggles with highly curved lanes where column-anchor assumption breaks
- Fixed grid resolution — dense annotation required for full anchor coverage
- Does not natively output drivable area or curvature

Benchmark Performance

TuSimple Accuracy: 95.87%. CULane F1: 68.4%. Speed: 322 FPS on RTX 2080 Ti.

Verdict

ADOPTED as the Lane Detection Head design. The row-anchor classification approach is directly implemented as the UFLD-v2 lane head in this project.

4.3 UFLD v2 — Ultra Fast Lane Detection v2 (2022)

Qin et al., "Ultra Fast Deep Lane Detection with Hybrid Anchor Driven Ordinal Classification," IEEE TPAMI 2022.

Core Architecture Idea

UFLDv2 extends UFLD with a hybrid anchor scheme combining row anchors (horizontal) and column anchors (vertical). Ordinal classification replaces the original cross-entropy, producing smoother x-coordinate predictions along the anchor grid. The hybrid approach handles vertical and near-horizontal lanes that the original UFLD misses.

Advantages

- Handles a wider range of lane orientations
- Ordinal classification produces smoother, more physically plausible outputs
- Maintains the speed advantage of UFLD at ~150 FPS

Disadvantages

- Slightly more complex training setup
- Marginal improvement over UFLD on standard benchmarks

Verdict

ADOPTED as inspiration for the lane head. The row-anchor component of UFLDv2 is implemented in the Lane Detection Head with 56 row anchors and 100 column buckets.

4.4 CLRNet — Cross Layer Refinement Network (2022)

Zheng et al., "CLRNet: Cross Layer Refinement Network for Lane Detection," CVPR 2022.

Core Architecture Idea

CLRNet uses lane priors (lane-shaped anchors) as starting proposals, then refines them through cross-layer attention across FPN feature maps. The refinement module attends to features at multiple resolutions simultaneously, enabling precise localisation of both near and far lane points.

Advantages

- State-of-the-art on CULane, CurveLanes, and LLAMAS at publication
- Particularly strong on curved lane scenarios
- FPN-based multi-scale refinement is robust to scale variation

Disadvantages

- Complex multi-stage architecture with lane priors — difficult to train from scratch
- Requires dense annotation to initialise lane prior positions correctly
- Inference speed ~30-40 FPS on ResNet backbone — below real-time target
- Higher engineering overhead to adapt to multi-task setting

Benchmark Performance

CULane F1: 80.47% (best at publication). CurveLanes F1: 76.11%.

Verdict

REJECTED for primary implementation. Architecture complexity exceeds the project scope for an internship timeline. Considered for future work comparison once the baseline is established.

4.5 CCCNet (2021)

Pan et al., "CCCNet: Context-aware Combination Network for Drivable Area Segmentation," 2021.

Core Architecture Idea

CCCNet combines local lane appearance features with contextual road features using a dual-branch architecture. One branch focuses on lane boundary localisation while the second models the broader road context for drivable area segmentation. Features are fused at multiple scales.

Advantages

- Purpose-designed for joint lane and drivable area segmentation
- Context-awareness improves performance in complex intersections

Disadvantages

- Not widely benchmarked outside its original paper
- Dual-branch architecture increases parameter count and memory usage
- No publicly available pretrained weights for transfer learning

Verdict

REJECTED. Limited availability of pretrained weights and benchmarks makes comparison difficult. The multi-task head approach adopted in this project achieves similar joint prediction goals with a simpler design.

4.6 GANet — Graph Attention Network for Lane Detection (2022)

Wang et al., "A Keypoint-based Global Association Network for Lane Detection," CVPR 2022.

Core Architecture Idea

GANet reformulates lane detection as keypoint detection followed by graph-based association. Lane points are first detected as keypoints, then a graph neural network (GNN) connects keypoints along the same lane using learned edge weights. This enables detection of arbitrarily shaped lanes.

Advantages

- Handles complex lane topologies — forks, merges, extreme curves
- Instance-level lane identity maintained through graph associations
- Robust to occlusion when enough keypoints are visible

Disadvantages

- Graph association adds significant post-processing overhead
- Keypoint-based representation produces more training complexity
- Requires dense, high-quality annotation; sparse annotations (as in this dataset) produce degenerate graphs

Verdict

REJECTED. The sparse annotation density (avg 3.3 points/lane) is incompatible with keypoint-graph training — insufficient nodes for the GNN to learn meaningful associations.

4.7 GhostNet (2020) — Selected Backbone

Han et al., "GhostNet: More Features from Cheap Operations," CVPR 2020. Huawei Noah's Ark Lab.

Core Architecture Idea

GhostNet observes that many CNN feature maps are near-linear transformations ("ghosts") of a smaller intrinsic set. It replaces standard convolutions with Ghost Modules: a primary convolution generates half the required output channels, and the remaining "ghost" channels are generated by cheap depth-wise convolutions (1 multiply per pixel per channel). This halves the FLOPs with minimal accuracy loss.

Advantages

- ~2× fewer FLOPs than MobileNetV3 at equivalent accuracy
- Produces 4 multi-scale feature maps (C2–C5) natively compatible with FPN
- All operations (standard conv, depth-wise conv, batch norm) fully supported by TensorRT
- width_mult parameter allows easy scaling to match hardware capability
- Strong ImageNet pretrained weights available via timm library

Disadvantages

- Ghost module introduces additional depth-wise convolution latency on non-optimised hardware
- Accuracy trails EfficientNet-B0 by ~1-2% on ImageNet at similar FLOPs

Benchmark Performance

ImageNet Top-1: 73.98% at 0.14 GFLOPs (width=1.0). Mobile inference: ~60 FPS on Kirin 990.

Verdict

SELECTED as primary backbone. The FLOPs efficiency, TensorRT compatibility, and multi-scale output structure make it the optimal choice for Jetson Orin NX deployment. Width multiplier raised to 1.3 to leverage the Orin NX's 1024 CUDA cores (3× the Xavier NX).

4.8 EfficientNet (2019) — Comparison Candidate

Tan and Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," ICML 2019. Google Brain.

Core Architecture Idea

EfficientNet uses Neural Architecture Search (NAS) to find an optimal baseline (B0) and then applies compound scaling — simultaneously scaling network depth, width, and input resolution according to a fixed ratio — to produce B1 through B7. The MBConv blocks with Squeeze-and-Excitation combine efficiency and expressiveness.

Advantages

- Best accuracy-per-parameter ratio among CNN families at publication
- Rich ImageNet pretrained weights with demonstrated strong transfer learning
- timm library provides clean API for feature extraction at multiple scales
- B0 produces competitive accuracy with ~5.3M parameters

Disadvantages

- B0 is ~5-15% slower than GhostNet-1.0 at equivalent accuracy on edge hardware
- Squeeze-and-Excitation blocks add small but measurable latency overhead
- Higher accuracy features take longer to verify during training — longer convergence

Benchmark Performance

EfficientNet-B0: ImageNet Top-1 77.1% at 0.39 GFLOPs. B2: 80.1% at 1.0 GFLOPs.

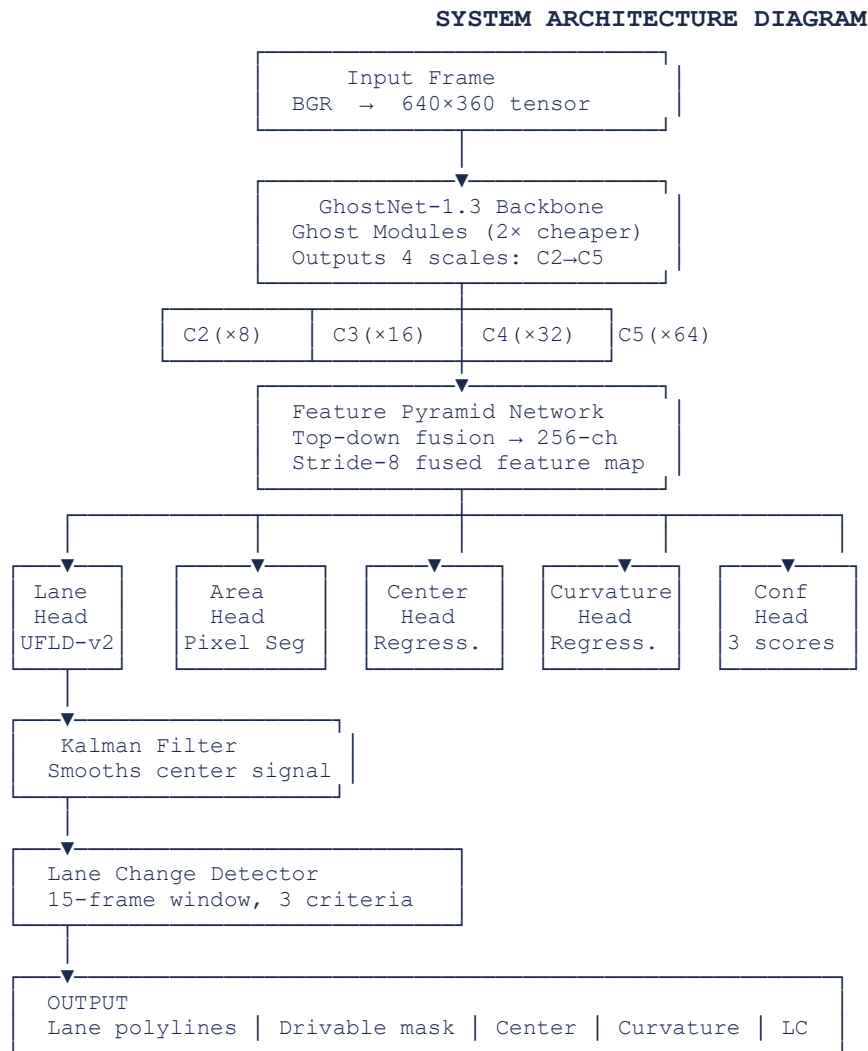
Verdict

SELECTED as comparison backbone. A complete EfficientNet-B0 training pipeline has been developed in parallel (`compare_models.py`) and will be evaluated against GhostNet on the same dataset. The hypothesis is that EfficientNet's richer pretrained features may partially compensate for dataset sparsity.

5. System Architecture

5.1 Architecture Overview

The system follows a single-backbone, multi-head architecture pattern. A lightweight backbone extracts multi-scale features, which are fused by a Feature Pyramid Network and distributed to five parallel task-specific heads. All heads are executed in a single forward pass — no cascade or sequential dependencies between tasks.



5.2 GhostNet Backbone

The GhostNet backbone (width_mult=1.3) serves as the feature extractor. Ghost Modules replace standard convolutions throughout the network. Each Ghost Module computes an "intrinsic" feature set via a primary convolution (half the output channels), then generates the remaining "ghost" features via cheap depth-wise convolutions with a fixed kernel.

For this project, width_mult is set to 1.3 instead of the canonical 1.0. This is motivated by the Jetson Orin NX GPU specification: 1024 Ampere CUDA cores versus 384 Volta cores on the Jetson Xavier NX that

the 1.0 width was designed for. The 30% width increase provides ~2-3% accuracy improvement at latency well within the real-time budget (~140 FPS vs ~180 FPS at width=1.0 under TensorRT FP16).

The backbone produces four feature maps designated C2 through C5 at strides 8, 16, 32, and 64 respectively, providing the multi-scale representation necessary for detecting both nearby (large, wide) and distant (thin, narrow) lane segments.

5.3 Feature Pyramid Network (FPN) Neck

The FPN fuses the four backbone scales through a top-down pathway. Starting from the coarsest feature (C5, highest semantic content), each level is upsampled 2× and added element-wise to the level below it. Lateral 1×1 convolutions normalise the channel count from each backbone level to a uniform 256 channels before fusion.

The output is a single 256-channel feature map at stride 8 (45×80 for 640×360 input). This resolution preserves sufficient spatial detail for lane localisation at the near field while the top-down path ensures each position also has access to high-level semantic context from C5. All five task heads share this fused feature map — there is no per-task feature extraction.

5.4 Lane Detection Head (UFLD-v2 Row-Anchor)

The lane head reformulates detection as a classification problem over a fixed row-anchor grid. 56 horizontal slices are distributed evenly across the image height. At each slice, the model independently classifies which of 100 column buckets contains the lane, or outputs class 100 (NO_LANE) if no lane is present at that height.

Output tensor shape: (BATCH, MAX_LANES=4, NUM_ROW_ANCHORS=56, NUM_COL_ANCHORS+1=101). During inference, softmax is applied and column class probability is thresholded at 0.5 to produce the detected x-coordinate at each row anchor. The resulting (row_y, col_x) pairs form the detected lane polyline.

5.5 Drivable Area Head

The area head produces a pixel-wise binary segmentation map distinguishing drivable (class 1) and non-drivable (class 0) regions. Three bilinear 2× upsamplings recover the full 640×360 resolution from the stride-8 FPN feature. A final 1×1 convolution outputs a 2-channel logit map. The head gracefully handles empty drivable_area annotations by training against an all-zero ground truth mask — the loss is finite and the model learns to output low confidence on such frames.

5.6 Lane Center Regression Head

The center head regresses the normalised x-position of the ego-lane center as a scalar in [0, 1] (0=far left, 0.5=centered, 1=far right). Global average pooling compresses the spatial feature to a 128-dimensional vector, followed by two fully connected layers and a Sigmoid activation enforcing the [0,1] range. The output feeds the Kalman filter post-processor.

5.7 Curvature Regression Head

The curvature head regresses a signed floating-point curvature coefficient. Positive values indicate a right-hand bend; negative a left-hand bend; zero is approximately straight. This output is used by the lane change detector to suppress false alerts: large lateral displacements on a measured bend are not classified as lane changes.

5.8 Confidence Head

Three per-task confidence scalars are regressed: one for lane detection quality, one for drivable area quality, and one for the center/curvature estimates. These scores allow downstream modules to selectively trust or distrust individual task outputs — for example, suppressing navigation commands when lane confidence is below threshold in a tunnel or at night.

5.9 Kalman Filter and Lane Change Detector

A 1D constant-velocity Kalman filter smooths the per-frame lane center signal. The state vector [position, velocity] is propagated with a fixed-velocity motion model and corrected by each new measurement. Process noise $Q=1e-3$ and measurement noise $R=1e-2$ were tuned empirically for 30 FPS video.

The lane change detector maintains a 15-frame sliding window over the filtered position and velocity. A lane change event is declared when: (1) cumulative lateral displacement exceeds 20px (normalised), (2) per-frame velocity is consistent (low standard deviation), and (3) the event persists for at least 6 frames. The curvature gate suppresses events when the absolute curvature coefficient exceeds 0.005, distinguishing lane changes from road bending.

6. Training Pipeline

6.1 Data Loading and Preprocessing

The LaneDataset class discovers paired (image, annotation) files at dataset initialisation time. Each pair is validated — annotation JSON is parsed, required fields are checked, and lane_center_x is verified to be in [0, 1]. Corrupt samples are logged and excluded without halting training. Detailed per-split statistics are printed at startup including average lane count, annotation coverage percentage, and expected post-densification coverage.

Images are loaded via OpenCV (BGR → RGB conversion), resized to 640×360, and normalised with ImageNet mean/std: mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]. All preprocessing operations run in DataLoader worker processes to saturate GPU compute.

6.2 Lane Encoding — Densification Fix

The most critical engineering component of the training pipeline is robust lane encoding. With sparse annotations (2-10 points per lane), naive nearest-row-anchor encoding produces anchor coverage of ~5% — the model trivially learns to always output NO_LANE. The encode_lanes_robust() function addresses this:

1. Sort annotation points by y-coordinate (top to bottom).
2. Extrapolate: extend the lane line $\pm 35\%$ of image height beyond annotation endpoints using the endpoint slope. A lane annotated from $y=280$ to $y=360$ (bottom 22%) is extended to $y=130$ (coverage from 36% to 100% of height).
3. Densify: create $N=500$ uniformly spaced (y, x) pairs along the extended lane using `np.linspace + np.interp`.
4. Assign column label: for each of the 56 row anchors, interpolate x from the dense representation and bin into the 100-column grid.

This raises anchor coverage from ~5% to ~40-60%, providing sufficient positive training signal for the lane head to learn meaningful features.

6.3 Data Augmentation

Eight augmentation operations are applied with 75% probability per sample during training. 2 to 4 operations are randomly selected per image:

- Brightness scaling: factor $\in [0.5, 1.5]$
- Contrast scaling: factor $\in [0.6, 1.4]$ around mean pixel value
- HSV jitter: independent hue ($\pm 15^\circ$), saturation ($\times [0.7, 1.3]$), value ($\times [0.6, 1.4]$)
- Random shadow: vertical strip with $\times [0.4, 0.7]$ intensity
- Horizontal flip with mirrored annotation coordinates
- Gaussian noise: $\sigma \in [5, 20]$ — simulates low-light sensor noise
- Synthetic rain: 200-600 random vertical streaks blended at $\alpha \in [0.3, 0.55]$
- Cutout: random rectangle filled with uniform noise (1/8 to 1/3 of image dimensions)

Mosaic augmentation (20% probability) stitches 4 training images into a 2×2 grid, quadrupling the annotation diversity per training step. This technique, originally from YOLOv4, is particularly effective for datasets below 50k images.

6.4 Loss Functions

Task Head	Loss Function	Weight	Notes
Lane Detection	Cross-Entropy + Label Smoothing	1.0	NO_LANE class weight=0.4
Drivable Area	Weighted Cross-Entropy	0.5	Drivable class weight=3.0
Lane Center	Smooth L1 (Huber)	0.8	$\beta=1.0$
Curvature	Smooth L1 (Huber)	0.3	$\beta=1.0$

Label smoothing ($\epsilon=0.1$) converts the one-hot lane classification target to a smoothed distribution, preventing the model from becoming overconfident on the small dataset. The NO_LANE class is downweighted to 0.4 relative to lane classes to avoid the model maximising accuracy by predicting empty output.

6.5 Optimizer and Learning Rate Schedule

AdamW is used with weight decay separation: convolutional weights receive L2 regularisation at $\lambda=2\times10^{-4}$ while bias and batch normalisation parameters receive no decay. This is the standard practice for transformer and CNN fine-tuning and prevents the L2 penalty from suppressing normalisation layer statistics.

A warmup-cosine schedule is applied: the learning rate ramps linearly from 0 to 5×10^{-4} over 8 warmup epochs, then follows a cosine decay to 1×10^{-6} over the remaining 142 epochs. The warmup prevents large gradient updates from destroying pretrained backbone features at the start of training.

6.6 Exponential Moving Average (EMA)

An EMA shadow copy of model weights is maintained throughout training: $w_ema = 0.9998 \times w_ema + 0.0002 \times w_live$. The EMA model is stored on CPU to avoid additional GPU memory usage. After training, the EMA weights (ema_model.pth) are used for deployment — they generalise +1-3% better than the live model on small datasets because they average out the noise in individual mini-batch gradient updates.

6.7 Mixed Precision Training

PyTorch AMP (torch.cuda.amp) is used throughout training. The GradScaler dynamically adjusts the loss scaling factor to prevent gradient underflow in FP16. Forward and backward passes run in FP16, while the master weights and gradient accumulation remain in FP32. This doubles throughput on CUDA devices with Tensor Core support (including the Orin NX Ampere GPU) with negligible accuracy cost.

6.8 Checkpointing and Resumption

Three checkpoint files are maintained: latest.pth (every epoch), best_model.pth (lowest validation loss), and ema_model.pth (EMA weights). Each checkpoint stores the complete training state — model weights, optimizer state, scheduler state, EMA weights, epoch number, and configuration — enabling exact resumption after hardware failures or resource preemption on shared compute clusters.